

# **TUGATAI AI SAAS PLATFORM**

## **FULL API CONTRACT SPECIFICATION DOCUMENT**

---

### **Prepared By**

**Md. Shahriar Hossain Jihad**

### **Version**

**Version 1.0**

### **API Architecture Style**

**REST API + WebSocket Hybrid Architecture**

### **Date**

**May 2026**

---

## **TABLE OF CONTENTS**

1. API Overview
2. API Standards
3. Authentication Strategy
4. Authorization & RBAC
5. Multi-Tenant API Strategy
6. Branch Isolation Strategy
7. Standard Response Format
8. Error Handling Standards
9. Pagination, Filtering & Sorting
10. Authentication APIs
11. Tenant APIs
12. Branch APIs
13. User & RBAC APIs
14. Omnichannel Inbox APIs

15. Message APIs
  16. CRM APIs
  17. Booking APIs
  18. Pricing Engine APIs
  19. AI APIs
  20. Knowledge Base APIs
  21. Workflow APIs
  22. Campaign APIs
  23. Analytics APIs
  24. Billing & Subscription APIs
  25. Notification APIs
  26. Upload APIs
  27. Webhook APIs
  28. Socket.IO Events
  29. Validation Rules
  30. HTTP Status Codes
  31. Security Standards
  32. API Best Practices
  33. Future API Scalability
  34. Conclusion
- 

## 1. API OVERVIEW

The Tugatai AI SaaS Platform uses:

- REST APIs for business operations
- Socket.IO for realtime communication
- Webhooks for third-party integrations
- BullMQ for async processing

The API architecture is designed for:

- modular monolith scalability
  - future microservice migration
  - multi-tenant SaaS operations
  - branch-based operations
  - realtime messaging
  - AI orchestration
-

## 2. API STANDARDS

### Base URL

```
/api/v1
```

---

### API Design Principles

The API follows:

- RESTful conventions
  - consistent response structures
  - versioned endpoints
  - modular route grouping
  - tenant-aware processing
  - branch-aware processing
  - stateless authentication
- 

### Content Type

```
Content-Type: application/json
```

---

## 3. AUTHENTICATION STRATEGY

### Authentication Method

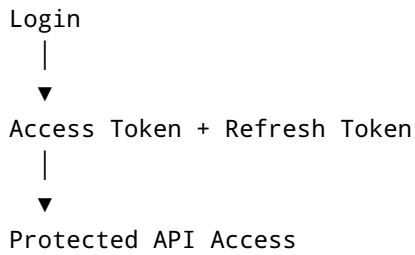
- JWT Access Tokens
  - Refresh Tokens
- 

### Authorization Header

```
Authorization: Bearer <token>
```

---

## Token Flow



## 4. AUTHORIZATION & RBAC

### Supported Roles

- Super Admin
- Tenant Owner
- Branch Manager
- Agent

### Access Control Rules

| Role           | Access Scope                   |
|----------------|--------------------------------|
| Super Admin    | Entire Platform                |
| Tenant Owner   | Tenant Resources               |
| Branch Manager | Assigned Branch                |
| Agent          | Assigned Conversations & Leads |

## 5. MULTI-TENANT API STRATEGY

### Tenant Isolation

Every protected API MUST:

- validate tenant access
- scope queries using tenant\_id
- prevent cross-tenant access

---

## Tenant Middleware

Middleware injects:

```
req.user  
req.tenant  
req.branch
```

from JWT.

---

## 6. BRANCH ISOLATION STRATEGY

### Branch Isolation

Branch-specific APIs MUST:

- validate branch access
  - filter branch resources
  - prevent unauthorized branch access
- 

### Branch Flow

```
Tenant  
|  
▼  
Branch  
|  
▼  
Users + Resources
```

---

## 7. STANDARD RESPONSE FORMAT

### Success Response

```
{  
  "success": true,
```

```
"message": "Operation successful",
"data": {},
"meta": {}
}
```

---

## Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": []
}
```

---

# 8. ERROR HANDLING STANDARDS

## Standard Error Structure

```
{
  "success": false,
  "message": "Unauthorized",
  "errors": [
    {
      "field": "email",
      "message": "Email is required"
    }
  ]
}
```

---

# 9. PAGINATION, FILTERING & SORTING

## Pagination

```
GET /api/v1/conversations?page=1&limit=20
```

## Filtering

```
GET /api/v1/leads?stage=hot
```

---

## Sorting

```
GET /api/v1/conversations?sort=created_at:desc
```

---

## Search

```
GET /api/v1/contacts?search=john
```

---

# 10. AUTHENTICATION APIs

## 10.1 Login

### Endpoint

```
POST /api/v1/auth/login
```

### Authentication

Public

### Request Body

```
{  
  "email": "admin@example.com",  
  "password": "password123"  
}
```

## Success Response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "access_token": "jwt-token",
    "refresh_token": "refresh-token",
    "user": {}
  }
}
```

---

## 10.2 Register

### Endpoint

```
POST /api/v1/auth/register
```

---

## 10.3 Refresh Token

### Endpoint

```
POST /api/v1/auth/refresh
```

---

## 10.4 Logout

### Endpoint

```
POST /api/v1/auth/logout
```



## 10.5 Forgot Password

### Endpoint

```
POST /api/v1/auth/forgot-password
```

---

## 10.6 Reset Password

### Endpoint

```
POST /api/v1/auth/reset-password
```

---

## 11. TENANT APIs

### 11.1 Create Tenant

```
POST /api/v1/tenants
```

---

### 11.2 Get Tenants

```
GET /api/v1/tenants
```

---

### 11.3 Get Tenant Details

```
GET /api/v1/tenants/:id
```

---

## 11.4 Update Tenant

```
PUT /api/v1/tenants/:id
```

---

## 11.5 Delete Tenant

```
DELETE /api/v1/tenants/:id
```

---

## 12. BRANCH APIs

### 12.1 Create Branch

```
POST /api/v1/branches
```

---

### 12.2 Get Branches

```
GET /api/v1/branches
```

---

### 12.3 Get Branch Details

```
GET /api/v1/branches/:id
```

---

### 12.4 Update Branch

```
PUT /api/v1/branches/:id
```

---

## 12.5 Delete Branch

```
DELETE /api/v1/branches/:id
```

---

## 13. USER & RBAC APIs

### 13.1 Create User

```
POST /api/v1/users
```

---

### 13.2 Get Users

```
GET /api/v1/users
```

---

### 13.3 Get User Details

```
GET /api/v1/users/:id
```

---

### 13.4 Update User

```
PUT /api/v1/users/:id
```

---

### 13.5 Delete User

```
DELETE /api/v1/users/:id
```

---

## 13.6 Assign Role

```
POST /api/v1/users/:id/roles
```

---

## 14. OMNICHANNEL INBOX APIs

### 14.1 Get Conversations

```
GET /api/v1/conversations
```

---

### 14.2 Get Conversation Details

```
GET /api/v1/conversations/:id
```

---

### 14.3 Assign Conversation

```
POST /api/v1/conversations/:id/assign
```

---

### 14.4 Close Conversation

```
POST /api/v1/conversations/:id/close
```

---

### 14.5 Add Conversation Tag

```
POST /api/v1/conversations/:id/tags
```

---

## 15. MESSAGE APIs

### 15.1 Send Message

```
POST /api/v1/messages
```

#### Request Body

```
{  
  "conversation_id": "uuid",  
  "message": "Hello",  
  "message_type": "text"  
}
```

### 15.2 Get Messages

```
GET /api/v1/conversations/:id/messages
```

### 15.3 Upload Media Message

```
POST /api/v1/messages/upload
```

## 16. CRM APIs

### 16.1 Create Lead

```
POST /api/v1/leads
```

## 16.2 Get Leads

```
GET /api/v1/leads
```

---

## 16.3 Get Lead Details

```
GET /api/v1/leads/:id
```

---

## 16.4 Update Lead Stage

```
PATCH /api/v1/leads/:id/stage
```

---

## 16.5 Add CRM Note

```
POST /api/v1/leads/:id/notes
```

---

## 16.6 Get CRM Activities

```
GET /api/v1/leads/:id/activities
```

---

## 17. BOOKING APIs

### 17.1 Create Booking

```
POST /api/v1/bookings
```

---

## 17.2 Get Bookings

```
GET /api/v1/bookings
```

## 17.3 Get Booking Details

```
GET /api/v1/bookings/:id
```

## 17.4 Update Booking Status

```
PATCH /api/v1/bookings/:id/status
```

## 17.5 Add Booking Items

```
POST /api/v1/bookings/:id/items
```

# 18. PRICING ENGINE APIs

## 18.1 Calculate Pricing

```
POST /api/v1/pricing/calculate
```

### Request Body

```
{  
  "country": "Qatar",  
  "weight": 10,  
}
```

```
"service_type": "air"  
}
```

---

## 18.2 Create Pricing Rule

```
POST /api/v1/pricing/rules
```

---

## 18.3 Get Pricing Rules

```
GET /api/v1/pricing/rules
```

---

## 18.4 Update Pricing Rule

```
PUT /api/v1/pricing/rules/:id
```

---

# 19. AI APIs

## 19.1 Generate AI Response

```
POST /api/v1/ai/respond
```

---

## 19.2 Summarize Conversation

```
POST /api/v1/ai/summarize
```



## 19.3 Extract Lead Data

```
POST /api/v1/ai/extract
```

---

## 19.4 Train AI Knowledge

```
POST /api/v1/ai/train
```

---

## 20. KNOWLEDGE BASE APIs

### 20.1 Upload Knowledge File

```
POST /api/v1/knowledge-base/upload
```

---

### 20.2 Get Documents

```
GET /api/v1/knowledge-base/documents
```

---

### 20.3 Delete Knowledge File

```
DELETE /api/v1/knowledge-base/documents/:id
```

---

### 20.4 Retrain AI Knowledge

```
POST /api/v1/knowledge-base/retrain
```

---

## 21. WORKFLOW APIs

### 21.1 Create Workflow

```
POST /api/v1/workflows
```

---

### 21.2 Get Workflows

```
GET /api/v1/workflows
```

---

### 21.3 Get Workflow Details

```
GET /api/v1/workflows/:id
```

---

### 21.4 Update Workflow

```
PUT /api/v1/workflows/:id
```

---

### 21.5 Execute Workflow

```
POST /api/v1/workflows/:id/execute
```

---

### 21.6 Get Workflow Executions

```
GET /api/v1/workflows/executions
```

---

## 22. CAMPAIGN APIs

### 22.1 Create Campaign

```
POST /api/v1/campaigns
```

---

### 22.2 Get Campaigns

```
GET /api/v1/campaigns
```

---

### 22.3 Get Campaign Details

```
GET /api/v1/campaigns/:id
```

---

### 22.4 Send Campaign

```
POST /api/v1/campaigns/:id/send
```

---

### 22.5 Pause Campaign

```
POST /api/v1/campaigns/:id/pause
```

---

### 22.6 Campaign Analytics

```
GET /api/v1/campaigns/:id/analytics
```

---

## 23. ANALYTICS APIs

### 23.1 Dashboard Analytics

```
GET /api/v1/analytics/dashboard
```

---

### 23.2 Conversation Analytics

```
GET /api/v1/analytics/conversations
```

---

### 23.3 Campaign Analytics

```
GET /api/v1/analytics/campaigns
```

---

### 23.4 Agent Analytics

```
GET /api/v1/analytics/agents
```

---

### 23.5 Branch Analytics

```
GET /api/v1/analytics/branches
```

---

## 24. BILLING & SUBSCRIPTION APIs

### 24.1 Create Subscription

```
POST /api/v1/billing/subscriptions
```

---

### 24.2 Get Subscription

```
GET /api/v1/billing/subscriptions
```

---

### 24.3 Cancel Subscription

```
POST /api/v1/billing/subscriptions/:id/cancel
```

---

### 24.4 Get Billing History

```
GET /api/v1/billing/payments
```

---

## 25. NOTIFICATION APIs

### 25.1 Get Notifications

```
GET /api/v1/notifications
```

---

## 25.2 Mark Notification Read

```
PATCH /api/v1/notifications/:id/read
```

---

## 26. UPLOAD APIs

### 26.1 Upload File

```
POST /api/v1/uploads
```

---

### 26.2 Delete File

```
DELETE /api/v1/uploads/:id
```

---

## 27. WEBHOOK APIs

### 27.1 Meta Webhook

```
POST /api/v1/webhooks/meta
```

---

### 27.2 Stripe Webhook

```
POST /api/v1/webhooks/stripe
```

---

## 27.3 OpenAI Callback Webhook

```
POST /api/v1/webhooks/ai
```

---

## 28. SOCKET.IO EVENTS

### Conversation Events

```
conversation:new  
conversation:update  
conversation:assigned  
conversation:closed
```

---

### Message Events

```
message:new  
message:typing  
message:read
```

---

### Lead Events

```
lead:new  
lead:update
```

---

### Notification Events

```
notification:new
```

---

## Campaign Events

```
campaign:started  
campaign:completed
```

---

## 29. VALIDATION RULES

### Validation Standards

All APIs MUST:

- validate request body
- validate query params
- validate path params
- validate tenant access
- validate branch access

---

### Required Validation Libraries

Recommended:

- Zod OR
- Joi

---

## 30. HTTP STATUS CODES

| Status Code | Meaning          |
|-------------|------------------|
| 200         | Success          |
| 201         | Resource Created |
| 400         | Bad Request      |
| 401         | Unauthorized     |
| 403         | Forbidden        |
| 404         | Not Found        |



| Status Code | Meaning               |
|-------------|-----------------------|
| 409         | Conflict              |
| 422         | Validation Error      |
| 500         | Internal Server Error |

---

## 31. SECURITY STANDARDS

### Security Requirements

All APIs MUST:

- use HTTPS
- validate JWT
- use RBAC
- sanitize inputs
- validate uploads
- rate limit requests
- log critical actions

---

### Rate Limiting

Recommended:

100 requests/minute per user

---

## 32. API BEST PRACTICES

### Required Standards

- consistent naming
  - modular route grouping
  - stateless APIs
  - reusable validation
  - service abstraction
  - repository abstraction
  - centralized error handling
-

## 33. FUTURE API SCALABILITY

### Future Scalability Goals

The API architecture supports:

- future microservices
- API gateways
- distributed services
- event-driven scaling

---

### Future Extraction Candidates

AI Service  
Campaign Service  
Analytics Service  
Workflow Engine  
Notification Service

---

## 34. CONCLUSION

The Tugatai AI SaaS Platform API architecture is designed as a scalable enterprise-grade backend contract system supporting:

- omnichannel communication
- CRM operations
- AI orchestration
- workflow automation
- booking systems
- campaign management
- SaaS scalability
- branch operations
- realtime messaging

The API contract prioritizes:

- scalability
- maintainability
- modularity
- security
- consistency

- future extensibility
- production readiness